Agentic Project Management

An enhancement of the emerging paradigm of
Spec-Driven Development

Research Study Participation Guide

Sub-Study B: GitHub Copilot

Conducted by:
Konstantinos Chaidemenos - CobuterMan

Institution:
National and Kapodistrian University of Athens



1. Welcome & Introduction

Thank you for participating in this research study. Your participation is essential for evaluating the Agentic Project Management (APM) framework for software development.

The goal of this study is to compare how APM performs on a practical, well-defined coding task against other AI-assisted frameworks. Please be assured that **you are not being tested**; the AI tools are. All data, including incomplete or non-functional code, is valuable to this research.

Data Privacy & Anonymity

All data that you will submit will be handled with strict confidentiality. Your name or any personal identifiers will not be linked to the data. The research artifacts that will be used for this academic research, will be anonymized in any resulting publications. You have been assigned a unique participant ID for this study, which you must include in your submission file names as described in the submission instructions.

All submitted chat logs will be encrypted using a secure encryption key before being stored in a publicly accessible dataset repository. This ensures that while the dataset structure is transparent, the actual content of your AI interactions remains confidential and cannot be read without the encryption key, which is held only by the research team. By proceeding, you consent to the anonymous collection and use of this data as described.

2. Study Overview

2.1. Your Task

Your task is to research an existing legacy PHP codebase, then implement a new feature in an existing Python codebase based on your findings and finally replicate the entire Python codebase in C for performance purposes. You will receive a detailed **Product Requirements Document (PRD)** as the sole source of truth for the specification, guiding both you and the AI tools.

2.2. Your Tools

You will complete this task **twice**, each time using a different AI-coding framework (e.g., APM, Spec-kit, Copilot's Plan agent). Sessions will be **time-boxed and isolated on separate branches**. Between sessions, you will submit your code changes, chat logs, and a brief survey.

> **Learning Effects Note**
>
> We acknowledge that performing the task twice may introduce learning effects. To mitigate this, some participants will begin with one tool, others with a different tool. This counterbalancing aims to ensure fair assessments across the study.

2.3. Your Time

Each session is strictly time-boxed to **three (3) hours**. It is important that you stop at the 3-hour mark, regardless of whether you are finished. Committing unfinished work is a perfectly valid and expected outcome.

2.4. Your Environment

To ensure the two sessions are completed in a controlled environment, make sure you meet the following requirements:

- **Operating System: Linux required** (e.g., Ubuntu 22.04+). The C replica implementation requires `valgrind` for memory leak testing, which is only available on Linux.
  - **Linux (native):** Ubuntu 22.04+ or similar distribution.
  - **Windows:** Windows Subsystem for Linux (WSL) with a recent Ubuntu distribution.
  - **macOS:** A Linux virtual machine or container (e.g., Docker with Ubuntu, or a Linux VM via VirtualBox/Parallels).
- **APM v0.5.3+:** Make sure you have the latest version of APM installed and initialized with the GitHub Copilot agent.
- **Workspace:** A dedicated directory with the following repositories cloned:
  - `eclass-mcp-server (8b4e39c)` - Main codebase where you will implement the new feature and create the C replica.
    After cloning, run: `cd eclass-mcp-server && git checkout 8b4e39c`. Then install dependencies using: `uv sync -dev -all-extras`.
  - `openeclass (e8b3329)` - Legacy PHP codebase where you will conduct research.
    After cloning, run: `cd openeclass && git checkout e8b3329`.
- **Python Version: 3.10+**
- **C Development Libraries:** Install the required C development libraries on your Linux system: `sudo apt-get install build-essential libcurl4-openssl-dev libxml2-dev valgrind`
  Critical: The grader expects minimum versions. Verify installation with:

```
gcc --version                       # Must be >= 9.0 (for C11 support)
pkg-config --modversion libcurl     # Must be >= 7.81.0
pkg-config --modversion libxml-2.0  # Must be >= 2.9.13
```

  Compatibility: Ubuntu 22.04 LTS (gcc 11, libcurl 7.81.0, libxml2 2.9.13) and Ubuntu 24.04 (gcc 13, libcurl 8.5.0, libxml2 2.9.14) both meet requirements.

- **IDE: Visual Studio Code** of at least version 1.106
- **AI Tool: GitHub Copilot**

Important Copilot Access Note

This study requires access to GitHub Copilot's premium features. Note that **verified students are eligible for the GitHub Education Pack**, which provides free access to GitHub Copilot Pro. You can sign up for the GitHub Education Pack for free at: https://education.github.com/pack

Please ensure you have access to GitHub Copilot's premium features in VS Code before beginning the session.

2.5. What You Will Deliver

At the end of each session, you will submit:

- A **submission zip file** containing a code patch and AI interaction logs. The file must follow the format: `{PID}_{S1|S2}_{platform}_{tool}_submission.zip` where:
  - `{PID}` is your assigned participant ID (e.g., `P001`)
  - `{S1|S2}` is the session number (`S1` for first session, `S2` for second session)
  - `{platform}` is `copilot` (lowercase)
  - `{tool}` is your assigned tool identifier: `apm`, `spec-kit`, or `copilot-plan` (lowercase, hyphenated if multi-word)

  Example: `P001_S1_copilot_apm_submission.zip` for Session 1 using APM. See Section 4.2 for instructions on creating the patch file and Section 5 for exporting AI interaction logs from Copilot.

- A brief **post-session survey** about your experience using each of the AI-coding frameworks. See Sections 4.2 and 4.3 for links to the surveys.

3. AI Tools: Setup & Instructions

Before starting your sessions, familiarize yourself with your assigned tools. Below are setup instructions and usage guidelines for each tool.

3.1. APM in VS Code (Copilot)

Description:

Agentic Project Management (APM) is designed to enhance your development process by decomposing specifications into actionable tasks and coordinating their execution across multiple AI agents; aiming to improve efficiency and context management.

Setup Instructions:

- Ensure you have `npm` installed on your system.
- Install the `agentic-pm` package globally using the following command:

```
npm install -g agentic-pm
```

- Once installed, use the `apm init` command within your working directory's root (alongside the other codebases) and select `GitHub Copilot`.

Video Tutorial & Familiarization:

To get acquainted with APM, visit the official documentation which includes video tutorials and getting started guides before starting your timed session.

3.2. Spec-kit in VS Code (Copilot)

Description:

Spec-kit is another AI-powered development framework designed to enhance the specification and planning stages of software development, ensuring a spec-first approach throughout the implementation process.

Setup Instructions:

- Ensure `uv` has been installed and set up on your system.
- Install the `specify-cli` using their official recommended installation command:

```
uv tool install specify-cli --from git+https://github.com/github/spec-kit.git
```

- Once installed, use the `specify init <your-project-name>` command within your working directory's root (alongside the other codebases) and select `GitHub Copilot`.

Video Tutorial & Familiarization:

To get acquainted with Spec-kit watch this tutorial and/or read the official documentation before starting your timed session.

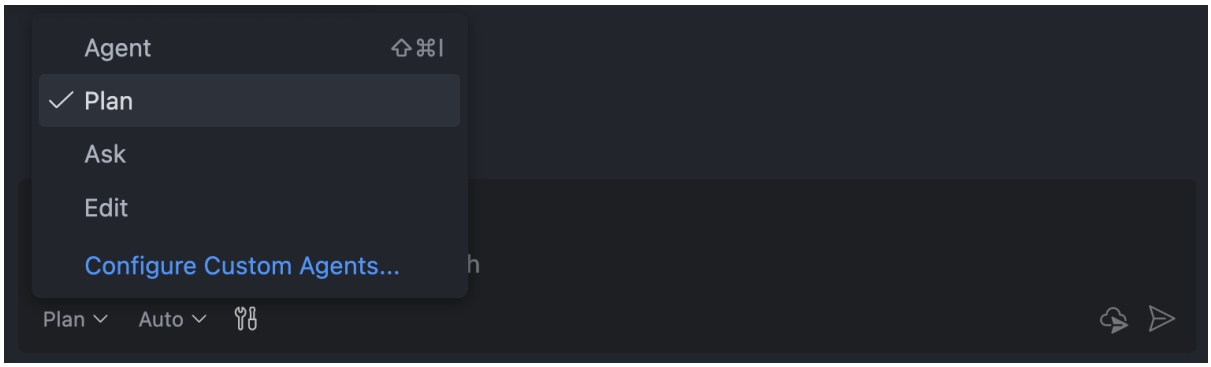
3.3. Plan agent in Copilot

Description:

Copilot's **Plan agent** implements Spec-Driven Development principles by emphasizing a "first-plan-then-execute" workflow. The agent is designed to break down the task into manageable steps, ask clarifying questions, and present a full implementation plan **before** any code is written.

Setup Instructions:

No additional setup is required beyond having VS Code with GitHub Copilot installed. To enable the Plan agent, select it from the agent picker of Copilot's Chat panel:



Video Tutorial & Familiarization:

The Plan agent is integrated directly into Copilot. For a brief overview, see Copilot's official documentation on the Plan agent.

Important Familiarization Note

Before starting your timed session, it is **crucial** to familiarize yourself with the features and workflows of your assigned tool by watching the provided tutorials and reviewing the relevant documentation. You are encouraged to practice using the tool on small, unrelated tasks to build confidence. However, **please avoid using the actual task or codebase during practice to minimize learning effects.**

This preparation will help you use the tool effectively during the task. In the post-session survey, you will be asked about your experience and familiarity with the tool.

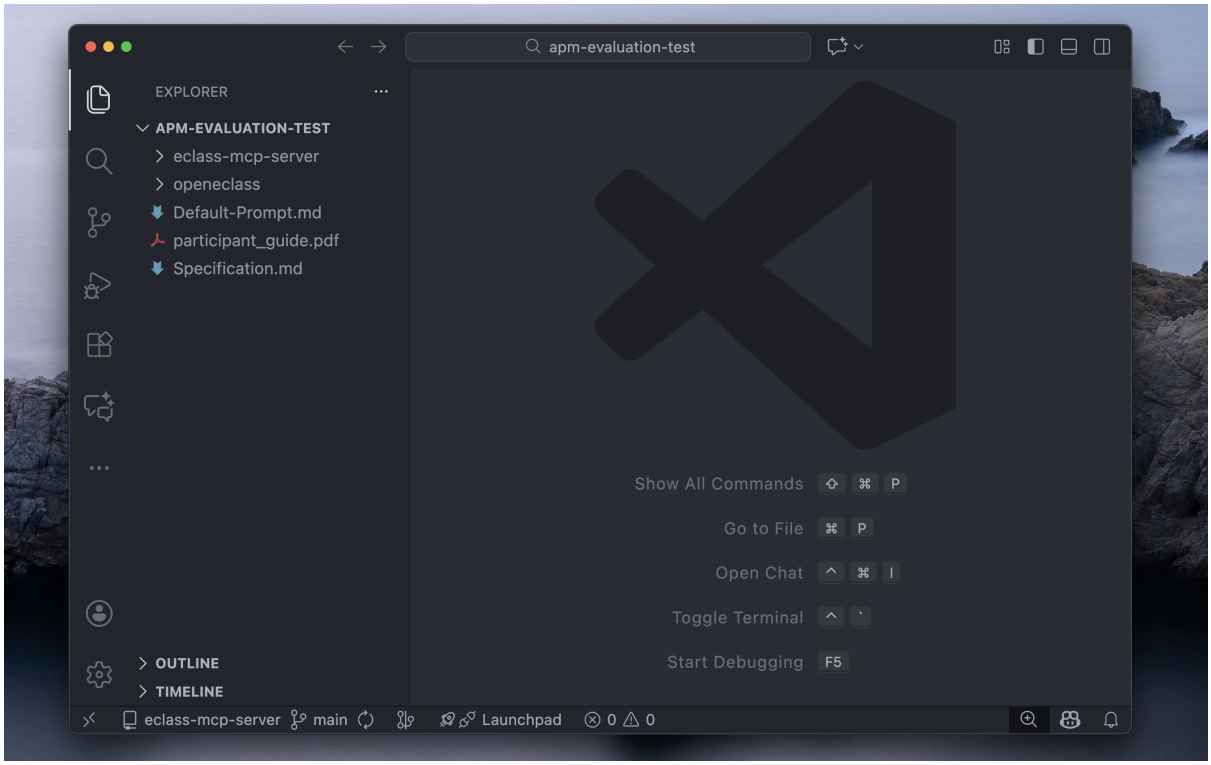
4. Session Workflow (Step-by-Step)

This section explains the session workflow in which you are to attempt to complete your task. Please follow these steps in order.

4.1. Ensure Environment is Set Up

Following 2.4, ensure you have installed VS Code with GitHub Copilot and that you have correctly set up your environment with all dependencies installed and the required repositories at the correct commit hashes. You may include the `Specification.md` and the `Default_Prompt.md` files alongside this guide in your workspace for your convenience.

Opening VS Code in your designated workspace directory would look like this:



Verify that all the base dependencies are installed by running:

```
/PATH/TO/PYTHON3 --version && uv --version && npm --version

cd eclass-mcp-server && /PATH/TO/PYTHON3 -c "import mcp, requests, bs4, dotenv"

gcc --version && make --version && valgrind --version

pkg-config --exists libcurl && pkg-config --exists libxml-2.0
```

4.2. Session 1: First Tool

Initialize Git and Create Branch:

Ensure you're starting from a clean state and create an isolated branch (replace `{tool}` with your assigned tool identifier, e.g., `apm`, `spec-kit`, or `copilot-plan`):

```
cd eclass-mcp-server                # go to implementation codebase root
git checkout main                   # switch to main branch
git checkout -b s1_{tool}           # create new branch for Session 1
```

Before you start working:

- Launch your first assigned AI tool following its setup instructions in Section 3.
- **Thoroughly read the** `Specification.md` (the PRD) provided alongside this guide.
- Use the **Default Prompt** provided alongside this guide to initiate your conversation with your AI tool.
- **Note the start time**; it will be used to track your session duration.
- Use **only** your assigned tool for AI assistance during this session.
- Start the timer and get to work!

> **Important Copilot Workflow Note**
>
> Due to Copilot's request-based billing, participants should follow a **Hybrid Model strategy** when using these tools. This is to conserve your request quota.
>
> – **APM sessions:**
>
> * **Setup Phase:** Use **Claude Sonnet 4** for the initial **Setup Agent** session.
> * **Task Loop Phase:** Keep using **Claude Sonnet 4** for the **Manager Agent** and switch to whichever non-premium model you prefer for the **Implementation Agents** throughout the task execution. Non-premium models are the ones with a 0x multiplier in the model picker.
>
> – **Plan Agent and Spec-kit sessions:** use **Claude Sonnet 4** for planning and non-premium for execution.

Work Until Completion or Time Limit:

- Iterate on the task for **up to 3 hours**.
- **Stop at the 3-hour mark**, even if you haven't completed the task.
- Alternatively, stop when you have fully completed the task **(if before 3 hours)**.

Create Code Patch:

After the 3 hour mark or upon completion, generate your solution patch for this session:

```
git add . && git commit -m "Session 1 solution"
git diff main s1_{tool} > {PID}_S1_copilot_{tool}_solution.patch
```

Git Note

Feel free to use Git for version control during your session. However, ensure that you **do not modify the main branch directly**. All code changes related to your task should be committed to your isolated session branch (`s1/2_toola/b`) before concluding the session.

After creating the patch file, **immediately save/move it to a safe location outside your repository**. Do not delete your branch until you have saved the patch file and exported all logs; these files will be lost if you delete the branch first.

Export AI Interaction Logs:

Follow the specific instructions in Section 5 to export chat conversations. Organize these in a `{PID}_S1_copilot_{tool}_logs/` folder (replace placeholders with your actual values).

Clean Up and Ensure Isolation:

Return to main branch and remove the solution branch:

```
git checkout main
git branch -D s1_{tool}
```

Important Branch Isolation Note

At this step, you will remove all changes made during Session 1 from your local repository. Ensure that you have successfully created the patch file, exported all logs and stored both in a safe location before proceeding, as these changes will no longer be recoverable.

Complete Session 1 Survey:

Fill out the brief Session 1 post-session survey.

Session 1 Checklist:

Before moving to Session 2, ensure you have completed the following:

- Created the patch file `{PID}_S1_copilot_{tool}_solution.patch` and deleted the session branch `s1_{tool}`.
- Exported and stored all AI interaction logs as `.json` chat files organized in `{PID}_S1_copilot_{tool}_logs/chats/` subfolder
- Completed the Session 1 post-session survey.

4.3. Session 2: Second Tool

Follow the same workflow as Session 1, but with the following adjustments:

- Create a branch named `s2_{tool}` from the clean `main` branch (replace `{tool}` with your assigned tool identifier for Session 2).
- Use your second assigned tool following its setup instructions in Section 3.
- After the 3 hour mark or upon task completion, generate your solution patch for this session:

```
git add . && git commit -m "Session 2 solution"
git diff main s2_{tool} > {PID}_S2_copilot_{tool}_solution.patch
```

- Export and store your AI interaction logs as `.json` chat files organized in `{PID}_S2_copilot_{tool}_logs/chats/` subfolder
- Complete the Session 2 post-session survey.

> **Important Session 2 Note**
>
> You may start Session 2 immediately after Session 1 or take a break. Ensure you are ready to approach Session 2 from scratch, maintaining a fresh perspective and minimizing any influence from Session 1. Before starting, please **make absolutely sure you have all the artifacts from session 1 checklist** safely stored.

Session 2 Checklist:

Before concluding your work, ensure you have completed the following:

- Created the patch file `{PID}_S2_copilot_{tool}_solution.patch` and deleted the session branch `s2_{tool}`.
- Exported and stored all AI interaction logs as `.json` chat files organized in `{PID}_S2_copilot_{tool}_logs/chats/` subfolder
- Completed the Session 2 post-session survey.

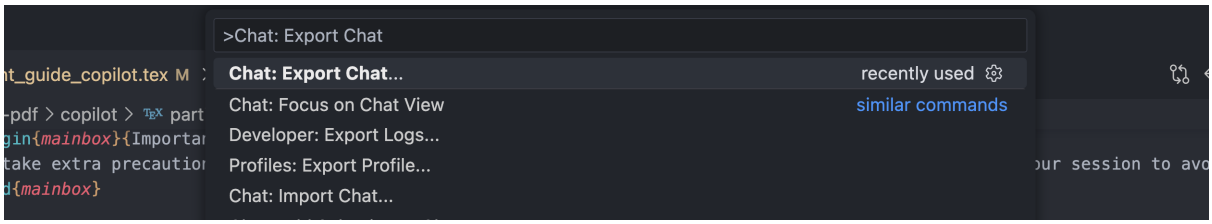
5. Exporting AI Interaction Logs from VS Code (Copilot)

To export your AI interaction logs from Copilot, follow these steps for both sessions.

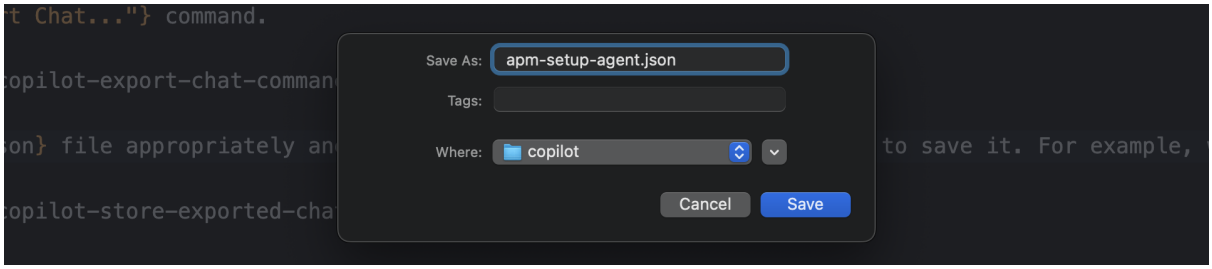
5.1. Export Chat Conversations

When using your assigned tools, you may open multiple chat conversations (e.g., at least three with APM: Setup, Manager, and Implementation Agents; one or two with Spec-kit or Plan agent). **Retain all chat conversations for our analysis. Do not delete any during your sessions**. To export chats, follow these steps:

- For each chat conversation opened during your session, **navigate to that specific chat** in the VS Code sidebar so it is your **active, visible chat tab**.
- Open the Command Palette (`Cmd+Shift+P` on macOS or `Ctrl+Shift+P` on Windows/Linux).
- Type and run the **"Chat: Export Chat..."** command.



- Save the exported `.json` file using the **strict naming format**: `{PID}_{S1|S2}_{platform}_{tool}_chat_{NN}.json` where:
  - `{PID}` is your participant ID (e.g., `P001`),
  - `{S1/2}` is the session number,
  - `{platform}` is `copilot`,
  - `{tool}` is your assigned tool identifier (e.g., `apm`, `spec-kit`, `copilot-plan`),
  - `{NN}` is the chat number starting from `01` (e.g., `P001_S1_copilot_apm_chat_01.json`, `P001_S1_copilot_apm_chat_02.json`).



Repeat this process for all conversations used during the session.

Important Chat Export Note

To take extra precautions, consider exporting conversations **immediately after completion** to avoid accidental deletions or technical issues. For example, in APM the Setup Agent conversation can be exported right after the Setup Phase is complete.

5.2. Organize Logs for Submission

Save the exported chat conversations in a dedicated logs folder. Create the following folder structure using your participant ID, session number, platform, and tool identifier:

```
{PID}_S1_copilot_{tool}_logs/
    chats/
        {PID}_S1_copilot_{tool}_chat_01.json
        {PID}_S1_copilot_{tool}_chat_02.json
        ...
```

6. Final Submission

After completing a session, organize your patch file and logs folder for submission.

6.1. Create Submission Zip Files

Once you have your patch file and organized logs folder for each session, create the final submission zip files using the correct naming format:

```
# For Session 1
zip -r {PID}_S1_copilot_{tool}_submission.zip
    {PID}_S1_copilot_{tool}_solution.patch {PID}_S1_copilot_{tool}_logs/

# For Session 2
zip -r {PID}_S2_copilot_{tool}_submission.zip
    {PID}_S2_copilot_{tool}_solution.patch {PID}_S2_copilot_{tool}_logs/
```

Important Submission Note

Ensure both the patch file and logs folder are at the root level of your zip file (not in subdirectories). The zip file should contain exactly two items: the `.patch` file and the `{PID}_{S1|S2}_copilot_{tool}_logs/` folder. Use the strict naming format `{PID}_{S1|S2}_{platform}_{tool}_submission.zip` as shown above. Only `{PID}` and `S` are uppercase; all other components must be lowercase.

After completing both sessions, submit all your data to our anonymous collection form.

Submission Checklist

Please be sure to upload the following single `.zip` files (one for each session):

- `{PID}_S1_copilot_{tool}_submission.zip`
  Contains: `{PID}_S1_copilot_{tool}_solution.patch` + `{PID}_S1_copilot_{tool}_logs/`
- `{PID}_S2_copilot_{tool}_submission.zip`
  Contains: `{PID}_S2_copilot_{tool}_solution.patch` + `{PID}_S2_copilot_{tool}_logs/`

7. Contact & Help

This study is conducted for a Bachelor's thesis by Kostantinos Chaidemenos - CobuterMan.

If you encounter any technical issues or have questions about the study procedure, contact:

```
sdi2200262@di.uoa.gr        # Email
cobuter_man                 # Discord
```

Thank you again for your time and contribution to this research.

